Can Small Language Models Learn Matrix Inversion?

A Three-Level Diagnostic Study of Syntactic, Structural, and Functional Learning via Supervised Fine-Tuning

MatrixInvLLM Experimental Report

May 15, 2026

Abstract

This report studies whether LoRA fine-tuning can improve a small language model's ability to map a matrix to its inverse when both input and output are represented as plain text. Using Low-Rank Adaptation ($r = 16$, $\alpha = 32$) applied to Llama-3.2-3B, the model is trained on 800 supervised (A, A^{-1}) examples covering 2×2 and 3×3 matrices, then evaluated on 100 held-out examples using a three-level diagnostic suite distinguishing syntactic validity, coarse numeric structure, and strict functional correctness.

The final evaluation does not show task-level success: the fine-tuned model still fails to produce correct inverses under matrix-residual checks ($\|AX - I\|_F < 10^{-1}$), and 40% of generations are unusable due to repeated decoding loops or malformed output shape.

At the same time, the fine-tuned model is not simply learning superficial formatting. Among its parseable outputs (60% of examples), several coarse diagnostics improve relative to the baseline: relative Frobenius error to the true inverse falls from 1.030 to 0.727, determinant-scale alignment rises from 0.045 to 0.639, sign agreement from 0.503 to 0.795, and spectral error from 5.015 to 3.829. These results suggest partial semantic learning at the level of global numeric structure, but not the precise functional behavior required for reliable matrix inversion.

Contents

1 Introduction **3**
1.1 Why Matrix Inversion? . . . 3
1.2 Experimental Approach . . . 3
1.3 Summary of Findings . . . 3

2 Methodology **4**
2.1 The Core Scientific Question . . . 4
2.2 Matrix Generation and Stability Criteria . . . 4
2.3 Data Serialization and Format . . . 4
2.4 Train/Val/Test Split . . . 5
2.5 Model and Training Setup . . . 5
2.6 Evaluation Framework . . . 5
2.6.1 Level 1 — Syntactic Metrics . . . 6
2.6.2 Level 2 — Coarse Numeric Diagnostics . . . 6
2.6.3 Level 3 — True Inverse Correctness . . . 6
2.6.4 Failure Mode Taxonomy . . . 7

3 Results **7**
3.1 Baseline Performance . . . 7
3.2 Fine-Tuned Performance . . . 8
3.3 Failure Mode Breakdown . . . 8
3.4 Comparative Interpretation . . . 8

4 Discussion **9**
4.1 Why the Functional Gap Persists . . . 9
4.2 Decoding Instability . . . 9
4.3 Interpretation . . . 10

5 Conclusion **10**

A Tables **12**

B Figures **13**

1 Introduction

Large language models (LLMs) have demonstrated strong capabilities across language understanding and generation. Their ability to perform *precise arithmetic reasoning* on structured numeric problems with verifiable answers is less well understood. A central question is whether supervised fine-tuning can teach a model to internalize a mathematical function, or whether it teaches the model only to produce outputs that resemble correct answers in format without satisfying the underlying constraints.

1.1 Why Matrix Inversion?

Matrix inversion provides a controlled benchmark for this question. Given a square matrix A, its inverse A^{-1} is the unique matrix satisfying:

$$A \cdot A^{-1} = I \qquad \text{and} \qquad A^{-1} \cdot A = I$$

where I is the identity matrix. The task has two useful properties. First, it is algorithmically demanding: computing an inverse requires multi-step Gaussian elimination with exact bookkeeping across all entries. Second, correctness is easy to verify via the residual norms $\|AX - I\|_F$ and $\|XA - I\|_F$, making ground truth unambiguous.

Even at 3×3 scale, the task is non-trivial for an autoregressive sequence model. The model must produce 9 floating-point values whose entrywise relationships jointly satisfy a global algebraic constraint, with no built-in access to numeric reasoning primitives. If a model sees many (A, A^{-1}) pairs during training, can it learn behavior consistent with approximate inverse computation? Or does it learn only surface regularities—output format, plausible value ranges—without satisfying the algebraic constraint?

1.2 Experimental Approach

To answer this question, this study uses a controlled synthetic setup with 1,000 stability-filtered matrix examples (2×2 and 3×3), deterministic text serialization, and LoRA fine-tuning of Llama-3.2-3B on 800 training examples.

The central question is not only whether the model can output a correct inverse, but also what kinds of intermediate structure it can learn when exact success is absent. For that reason, the evaluation separates three levels of behavior: syntactic validity, coarse numerical resemblance to the true inverse, and strict functional correctness under residual tests. This distinction matters because a model can fail at the end task while still revealing partial dependence on the underlying mapping from matrix to inverse.

1.3 Summary of Findings

The baseline model generates parseable matrix-like outputs consistently, but those outputs are numerically wrong under all task-level checks. Fine-tuning improves several coarse diagnostics among parseable outputs—determinant-scale alignment, sign agreement, and relative Frobenius error all improve noticeably—but does not achieve task-level correctness. Fine-tuning also introduces substantial reliability failures: only 60% of fine-tuned outputs are parseable, with 40% failing via repetition loops or malformed shapes.

The result is best described as partial semantic movement toward the target distribution without robust algorithmic success. Section 2 details the experimental setup. Section 3 presents the evaluation results. Sections 4–**??** interpret the findings and outline future directions.

2 Methodology

2.1 The Core Scientific Question

Our central hypothesis is that matrix inversion is a canonical example of a computation that is hard to produce algorithmically but easy to verify. A candidate inverse X for input A can be checked via the residual $\|AX - I\|_F$, making correctness unambiguous. The question we investigate is whether supervised fine-tuning can induce behavior consistent with approximate inverse computation in this setting—or whether the model learns only surface regularities such as output format and value ranges without satisfying the algebraic constraint.

We use a controlled synthetic setup to isolate this question from confounds such as ambiguous ground truth, label noise, or distribution mismatch.

2.2 Matrix Generation and Stability Criteria

We generate synthetic $n \times n$ matrices ($n \in \{2, 3\}$) by uniform random sampling from $[-1, 1]$, followed by stability filtering. To avoid ill-posed or numerically fragile inverse targets, we apply three criteria:

1. **Determinant threshold**: $|\det(A)| > 0.05$. This threshold ensures full rank and avoids near-singular matrices whose inverses have extremely large entries; below this threshold, numerical precision in the target serialization would be insufficient to distinguish from degenerate cases.

2. **Condition number bound**: $\kappa(A) < 50$. A condition number above this value implies that small perturbations in input encoding can produce large changes in the inverse, making the mapping harder to learn reliably. Both thresholds were chosen as conservative heuristic bounds based on standard numerical stability considerations.

3. **Residual validation**: The computed gold inverse X^* must satisfy $\|AX^* - I\|_F < 10^{-8}$, confirming that our ground truth is numerically sound.

We sample up to 20,000 candidates per matrix size and retain 500 examples per size, for a total dataset of 1,000 examples.

2.3 Data Serialization and Format

Inputs and targets are serialized in a deterministic machine-readable text format. A representative example:

```
MatrixSize: 3
Matrix:
0.39195, -0.18209, -0.65341
-0.68713, -0.49951, 0.09845
0.42919, 0.32039, -0.44013
```

The target output:

```
Inverse:
1.50708, -2.31690, -2.75565
-2.08222, 0.86379, 3.28444
-0.04613, -1.63052, -2.56829
```

Design rationale: (1) Strict, exact headers (`MatrixSize:`, `Matrix:`, `Inverse:`) are required by the parser, enabling reliable detection of missing or malformed blocks; (2) Row-major, comma-separated ordering makes structural errors detectable; (3) Fixed 5-decimal precision balances floating-point accuracy with token length.

At inference time, the prompt is appended with the open header `"\nInverse:\n"` so the model generates numeric rows directly, without needing to produce the header itself. Model outputs are post-processed by a truncation step that retains only the first valid `Inverse:` block, discarding any subsequent repetitions—this bounds the amount of looped content that reaches the parser. One known tradeoff is that strict header requirements may cause valid outputs to be classified as format failures if the model produces correct values with non-standard delimiters.

2.4 Train/Val/Test Split

We use a reproducible 80/10/10 split (seed = 123) across the full 1,000-example dataset:

- **Train**: 800 examples
- **Validation**: 100 examples
- **Test**: 100 examples (48 of size 2×2, 52 of size 3×3)

The split is randomized across both matrix sizes jointly; it is not stratified separately by size. The test-set composition of 48 and 52 examples reflects the random split outcome, not a deliberate allocation. All reported results are on the held-out test set.

2.5 Model and Training Setup

Base model: Llama-3.2-3B [3], a 3-billion-parameter decoder-only model.

LoRA configuration:

- Rank: $r = 16$
- Alpha: $\alpha = 32$ (effective scaling factor $\alpha/r = 2$)
- Dropout: 0.05

Optimization:

- Optimizer: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
- Learning rate: 5×10^{-5} with 5% linear warmup
- Weight decay: 0.01
- Training steps: 800 (batch size 16; 800 training examples are seen approximately once per step on average, giving roughly 1 epoch)
- Infrastructure: Tinker SDK with async pipelining

Inference: Greedy decoding (temperature = 0.0). Stop sequences include `<|eot_id|>`, `\nInverse:`, and `\n\nInverse:`, which terminate generation when the model begins a second inverse block. The inference service imposed an effective ceiling of 4,096 tokens; as discussed in §3, all fine-tuned model outputs reached this ceiling. No hyperparameter search was performed; this is an intentionally minimal configuration.

2.6 Evaluation Framework

This paper is a diagnostic case study of one specific training configuration, not a comprehensive benchmark of LLM algorithmic capability. Results should be interpreted in that scope.

2.6.1 Level 1 — Syntactic Metrics

- **Parse success rate**: Percentage of outputs parseable into a valid matrix.
- **Shape correctness**: Among parseable outputs, percentage with correct dimensions ($n \times n$).

2.6.2 Level 2 — Coarse Numeric Diagnostics

Important caveat: All Level 2 metrics are computed only on parseable (correct-shape) outputs. Baseline and fine-tuned Level 2 comparisons are therefore conditional on the respective parse-success subsets (100% of examples for the baseline, 60% for the fine-tuned model). Improvements in these metrics should be interpreted as improvements within the parseable subset, not as unconditional population-level improvements.

Metrics are averaged across all parseable examples pooled together (not stratified by matrix size unless noted).

- **Relative Frobenius error**: $\|X - A^{-1}\|_F / \|A^{-1}\|_F$. Values near 0 indicate predictions close to the true inverse; a value near 1.0 indicates the error is roughly equal to the magnitude of the target itself.
- **Determinant ratio**: $\det(X)/\det(A^{-1})$. A perfect inverse satisfies $\det(A^{-1}) = 1/\det(A)$, so a ratio of 1.0 indicates correct determinant scale. A ratio near 0 indicates poor alignment with the target determinant scale.
- **Sign agreement**: Fraction of entries where $\text{sign}(X_{ij}) = \text{sign}(A^{-1}_{ij})$. A value near 0.5 is expected for random predictions over balanced distributions; higher values indicate improved alignment of predicted sign structure with the gold inverse.
- **Spectral error**: Largest singular value of $(X - A^{-1})$. Captures the magnitude of the dominant error direction, complementing the Frobenius norm.
- **Copy-detection distances**:
  - $\|X - A\|_F$: Distance from the prediction to the input matrix.
  - $\|X - A^T\|_F$: Distance from the prediction to the transpose of the input.
  - $\|X - (-A)\|_F$: Distance from the prediction to the negation of the input.

  These detect whether the model is producing a trivial transformation of its input rather than computing the inverse. Large values indicate the prediction is not a simple copy, though they must be read alongside other metrics to characterize what the prediction actually contains.

2.6.3 Level 3 — True Inverse Correctness

- **Left residual**: $\|AX - I\|_F$. Primary success criterion.
- **Right residual**: $\|XA - I\|_F$. Checks both-sided inverse consistency for square matrices; for an exact inverse, both residuals are zero.
- **Tolerance-based accuracy**:
  - **Strict**: $\|AX - I\|_F < 10^{-2}$
  - **Medium**: $\|AX - I\|_F < 5 \times 10^{-2}$
  - **Loose**: $\|AX - I\|_F < 10^{-1}$

Passing even the loose tolerance would constitute evidence of approximate inverse behavior in this setup. All three failing simultaneously indicates the output is far from satisfying $AX \approx I$.

2.6.4 Failure Mode Taxonomy

We classify each output into one of six categories. Categories were defined a priori; some (EOS_ONLY, PROMPT_ECHO) were not observed in the reported run. When an output could in principle fit multiple categories, assignment follows the order below, with earlier categories taking precedence:

- `EOS_ONLY`: Output is empty after special-token removal.
- `PROMPT_ECHO`: Output mirrors the input prompt verbatim.
- `REPEATED_BLOCK_LOOP`: Output enters repetition loops and exhausts the token budget without a complete matrix.
- `SHAPE_FORMAT`: Output is partially parseable but has incorrect dimensions.
- `NUMERICALLY_WRONG`: Correct shape, but residual $> 10^{-1}$ (loose tolerance failure).
- `SUCCESS`: Residual $< 10^{-2}$ (strict tolerance pass).

This taxonomy distinguishes *decoding failures* from *numeric failures*, making it possible to separate whether the bottleneck is generation reliability or mathematical computation.

3 Results

The final evaluation shows a mixed outcome. The baseline model generated parseable outputs consistently, but those outputs remained numerically incorrect under all task-level checks. LoRA fine-tuning improved several diagnostics associated with the global structure of the target inverse distribution, yet it also introduced substantial reliability failures, especially repeated-block loops and malformed output shapes. The resulting picture is not one of simple success or simple failure, but of partial numeric improvement coupled with degraded generation stability.

All Level 2 diagnostics in this section are computed over parseable outputs only: all 100 baseline examples, and 60 fine-tuned examples. Level 3 correctness checks are unconditional over all 100 examples. See Table 3 for the full comparison and Table 4 for the fine-tuned failure mode distribution.

3.1 Baseline Performance

The baseline model performed better at surface-level generation than might be expected of an untrained model. Its outputs were parseable and correctly shaped on 100% of test examples, indicating that the base model already had sufficient exposure to the serialization format to produce matrix-like text without task-specific training.

However, these outputs were uniformly wrong at the mathematical level. Residual-based checks failed on every example: average $\|AX - I\|_F = 1.835$ and $\|XA - I\|_F = 1.851$, well above any acceptable tolerance. All 100 outputs were classified `NUMERICALLY_WRONG`. Coarse diagnostics confirm the same picture: the determinant ratio of 0.045 indicates little alignment with target determinant scale, and sign agreement of 0.503 is consistent with a near-random sign pattern relative to the true inverse.

This baseline behavior matters for interpretation. Because the model already produced well-formed matrices without fine-tuning, the primary contribution of training cannot be described

as teaching the model to emit structured output. The relevant question is whether fine-tuning moved the model closer to the inverse distribution in a numerically meaningful sense.

3.2 Fine-Tuned Performance

The fine-tuned model showed measurable improvement on several coarse numerical diagnostics among its parseable outputs. Relative error to the true inverse decreased ($1.030 \rightarrow 0.727$), determinant-scale alignment improved ($0.045 \rightarrow 0.639$), sign agreement increased ($0.503 \rightarrow 0.795$), and spectral error fell ($5.015 \rightarrow 3.829$). Residuals also improved among parseable examples ($\|AX - I\|_F$: $1.835 \rightarrow 1.719$; $\|XA - I\|_F$: $1.851 \rightarrow 1.480$). These shifts indicate that parseable fine-tuned outputs became more aligned with broad structural properties of the target inverse distribution.

These gains did not translate into task-level correctness. Among all 100 test examples, no fine-tuned output satisfied $\|AX - I\|_F < 10^{-1}$. Accuracy at strict, medium, and loose tolerances remained 0%.

Generation reliability also degraded. Only 60% of fine-tuned outputs were parseable and correctly shaped, compared to 100% for the baseline. Every fine-tuned output reached the 4,096-token inference ceiling, compared to typical baseline outputs of 52–55 tokens. Fine-tuning disrupted the model's length-control behavior: instead of producing a compact matrix and terminating, the model over-generated on every example. By matrix size, parse success was 70.8% for 2×2 and 50.0% for 3×3 examples.

3.3 Failure Mode Breakdown

The updated error profile reveals qualitatively different failure modes between the baseline and fine-tuned model. The baseline produced only `NUMERICALLY_WRONG` outputs—parseable matrices with wrong values. The fine-tuned model retained this failure mode (60 examples, 60%) but introduced two new categories:

- `REPEATED_BLOCK_LOOP` (22 examples, 22%): generation became trapped in local repetition and exhausted the token budget without completing a valid matrix.
- `SHAPE_FORMAT` (18 examples, 18%): outputs were parseable but had incorrect dimensions.

This distinction is important. A numerically wrong but parseable matrix can still reveal partial learning of the target mapping. By contrast, a repeated loop or malformed shape is primarily a generation-stability failure, and it prevents downstream evaluation of mathematical behavior entirely.

3.4 Comparative Interpretation

Taken together, the baseline and fine-tuned results suggest a layered interpretation. The baseline already handled the surface format of the task but failed to capture the inverse mapping. Fine-tuning improved coarse numeric structure among parseable outputs without producing dependable inverse computation. At the same time, it destabilized generation enough that 40% of outputs are excluded from numeric evaluation entirely.

The most defensible conclusion is that fine-tuning induced partial semantic movement toward the target distribution—not robust algorithmic success. This is stronger than saying the model learned only output syntax, yet weaker than claiming that the model learned matrix inversion in any reliable sense. The results indicate a separation between structural resemblance and functional correctness that is not bridged by the present training configuration.

Level	Baseline	Fine-Tuned
Syntactic (parse + shape, unconditional)	100.0%	60.0%
Structural (sign agreement, parseable only)	50.3%	79.5%
Functional (any tolerance pass, unconditional)	0.0%	0.0%

4 Discussion

The revised results complicate any simple interpretation. It is not accurate to say that the model learned only output formatting and no mathematical relationship between input and output. Several diagnostics improved in ways that suggest the fine-tuned model captured broad numerical regularities associated with the inverse, including scale, sign structure, and reduced similarity to the input matrix itself. At the same time, those improvements remain insufficient for task success. Matrix inversion is an unforgiving functional target: small entrywise deviations can yield large residual errors, and coarse resemblance does not guarantee algebraic correctness. The results point to a gap between approximate structural alignment and exact functional computation.

4.1 Why the Functional Gap Persists

Several plausible, non-exclusive explanations may account for the remaining gap. These explanations are not fully separable without further ablation experiments.

1. **Insufficient training signal.** 800 examples and 800 steps may be insufficient for a 3-billion-parameter model to generalize a precise function. Matrix inversion requires entrywise precision across all 4 or 9 output values; a small error in any entry can propagate into a large residual.

2. **Representation mismatch.** LLMs process token sequences without intrinsic numeric semantics. Each floating-point value is tokenized as a sequence of character-level tokens. The model must implicitly treat these as numeric quantities and compose them correctly—a form of reasoning not supported by standard language modeling objectives.

3. **Absence of intermediate computation.** Matrix inversion by Gaussian elimination proceeds through structured intermediate steps. Supervised fine-tuning on final (A, A^{-1}) pairs provides no signal about these steps, requiring the model to learn a direct input-to-output mapping that may be beyond what a small model can approximate from examples alone.

4. **Selection effects in Level 2 diagnostics.** The structural improvements reported in Section 3 are measured over the 60% of fine-tuned outputs that are parseable. These may not be representative of the full fine-tuned distribution: outputs that terminated cleanly enough to be parsed may systematically differ from those that looped or became malformed. The apparent improvements could be overstated if loop-prone examples would have shown weaker numeric alignment.

4.2 Decoding Instability

Format instability remained a primary bottleneck in this evaluation. All 100 fine-tuned outputs reached the 4,096-token inference ceiling, compared to typical baseline outputs of 52–55 tokens. The model lost clean length-controlled termination on every example after fine-tuning.

One possible explanation is that LoRA updates pushed the model toward generating more structured numeric content, and this effort leads to over-generation when the model cannot

converge to a well-formed output—producing repeated blocks or fragmented shapes. This explanation is speculative; the precise cause is not established by the current evaluation.

The practical consequence is significant. A model that shows improved numeric structure among parseable outputs, but generates parseable output on only 60% of examples, provides no net improvement in aggregate correctness (0 successes in both cases). Format instability must be addressed before the numeric improvements are meaningful in any applied sense.

4.3 Interpretation

A useful summary is to distinguish three levels of competence. First, a model may produce text that is syntactically well formed. Second, it may produce numbers that reflect some broad properties of the target distribution. Third, it may produce outputs that satisfy the mathematical function being asked. In this study, LoRA fine-tuning appears to improve the second level without reliably achieving the third, while also degrading robustness at the first level for 40% of test examples. This result is informative: it demonstrates that the gap between structural resemblance and functional correctness is real and not closed by this training approach.

The results are compatible with a capacity-limitation interpretation—a sufficiently large model or more targeted training might succeed—but they do not establish it. The failure could equally reflect a more fundamental limitation of text-based arithmetic supervision.

5 Conclusion

We investigated whether small language models can learn matrix inversion through supervised LoRA fine-tuning on synthetic data—a diagnostic case study, not a comprehensive algorithmic benchmark. Our three-level evaluation framework (syntactic, structural, functional) reveals a nuanced outcome.

What succeeded: Among parseable fine-tuned outputs, several coarse numeric diagnostics improved relative to the baseline. The determinant ratio rose from 0.045 to 0.639, sign agreement from 0.503 to 0.795, and relative Frobenius error to the gold inverse fell from 1.030 to 0.727. Parseable fine-tuned outputs were also substantially farther from trivial input copying ($\|X - A\|_F$: 1.45 $\rightarrow$ 3.18). These shifts indicate partial inverse-like structure in the fine-tuned model's parseable outputs, going beyond mere format imitation.

What failed: True inverse accuracy remained 0% under all residual-based tolerances for both models. Fine-tuning also introduced severe generation instability: only 60% of fine-tuned outputs were parseable (vs. 100% baseline), with exactly half of 3×3 examples failing, distributed between repetition loops (22%) and malformed shapes (18%).

Dominant bottlenecks: The experiment reveals two compounding issues. First, *insufficient numeric precision*: the model shows improved coarse alignment but cannot satisfy the algebraic constraint $AX \approx I$ at any tolerance. Second, *generation instability*: every fine-tuned output reached the 4,096-token inference ceiling, indicating a breakdown in length-controlled termination that prevents reliable evaluation of numeric quality. Addressing reliability is a prerequisite before further numeric progress can be meaningfully assessed.

Caveats: All structural improvements are measured conditionally over the 60% of fine-tuned outputs that are parseable; direct comparison with the fully parseable baseline is not strictly equivalent. Results represent a single training run without variance analysis.

Outlook: This result is a diagnostic, not a verdict. Future work should prioritize: (1) simplifying the output serialization format to reduce format-induced failures; (2) decoding stabilization to address looping; (3) small-scale overfitting experiments to separate model capacity limits from format and reliability bottlenecks; and (4) intermediate-step supervision that provides signal about the computation process, not only the final answer. Scaling experiments, model comparisons, and hyperparameter search are best pursued after reliability is established.

References

[1] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training verifiers to solve math word problems*. arXiv preprint arXiv:2110.14168.

[2] Hu, E. Q., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. arXiv preprint arXiv:2106.09685.

[3] Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., & others (2024). *The Llama 3 herd of models*. arXiv preprint arXiv:2407.21783.

A Tables

Table 1: Stability Criteria and Dataset Generation Statistics

Parameter	**Value**
Matrix sizes	2×2 and 3×3
Samples per size (target)	500
Total dataset size	1,000 examples
Train / Val / Test split	800 / 100 / 100
Test set composition	48 of size 2×2; 52 of size 3×3
Random distribution	Uniform $[-1, 1]$
Determinant threshold	$\lvert\det(A)\rvert > 0.05$
Condition number bound	$\kappa(A) < 50$
Residual tolerance (gold)	$\lVert AX - I\rVert_F < 10^{-8}$
Max generation attempts	20,000 per size
Decimal precision (serialization)	5
Split seed	123

Table 2: Training Configuration and Hyperparameters

Component	**Configuration**
Base model	Llama-3.2-3B
Adapter method	LoRA (Low-Rank Adaptation)
LoRA rank (r)	16
LoRA alpha (α)	32
Effective scaling (α/r)	2.0
Dropout	0.05
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
Learning rate	5×10^{-5}
Weight decay	0.01
Warmup ratio	5%
Training steps	800
Batch size	16
Training examples	800 (all from train split)
Max sequence length	2,048 tokens
Inference decoding	Greedy (temperature = 0.0)
Effective token ceiling (observed)	4,096 tokens
Training infrastructure	Tinker SDK (async pipelining)

Table 3: Diagnostic Comparison: Baseline vs. Fine-Tuned on Test Set ($n = 100$; 48 of size 2×2, 52 of size 3×3). Rows marked † are computed only over parseable outputs: all 100 baseline examples, 60 fine-tuned examples. Rows without † are unconditional over all 100 examples. ↑ = higher is better; ↓ = lower is better.

Metric	**Baseline**	**Fine-Tuned**	**Delta**
Generation Reliability (unconditional)			
Parse / shape rate	100.00%	60.00%	−40.00 pp
True Inverse Correctness (unconditional)			
Accuracy — strict ($\|AX - I\|_F < 10^{-2}$)	0.0%	0.0%	±0.0 pp
Accuracy — medium ($< 5 \times 10^{-2}$)	0.0%	0.0%	±0.0 pp
Accuracy — loose ($< 10^{-1}$)	0.0%	0.0%	±0.0 pp
Avg $\|AX - I\|_F \downarrow$ †	1.835	1.719	−0.116
Avg $\|XA - I\|_F \downarrow$ †	1.851	1.480	−0.371
Coarse Numeric Structure († parseable only)			
Avg relative $\|X - A^{-1}\|_F / \|A^{-1}\|_F \downarrow$ †	1.030	0.727	−0.303
Avg determinant ratio ↑ †	0.045	0.639	+0.594
Avg sign agreement ↑ †	0.503	0.795	+0.292
Avg spectral error ↓ †	5.015	3.829	−1.186
Input-Copy Proximity († parseable only)			
Avg $\|X - A\|_F$ †	1.448	3.175	+1.728
Avg $\|X - A^T\|_F$ †	1.585	2.760	+1.175
Avg $\|X - (-A)\|_F$ †	2.119	3.960	+1.841

Table 4: Fine-Tuned Model Failure Mode Breakdown ($n = 100$ test examples). The baseline produced no looping or shape failures; all 100 baseline outputs were classified `NUMERICALLY_WRONG`. The categories `EOS_ONLY` and `PROMPT_ECHO` were defined in the taxonomy but not observed in either run.

Failure Mode	**Count**	**Percent**
`NUMERICALLY_WRONG`	60	60.0%
`REPEATED_BLOCK_LOOP`	22	22.0%
`SHAPE_FORMAT`	18	18.0%
`SUCCESS`	0	0.0%
Total	100	100.0%

B Figures

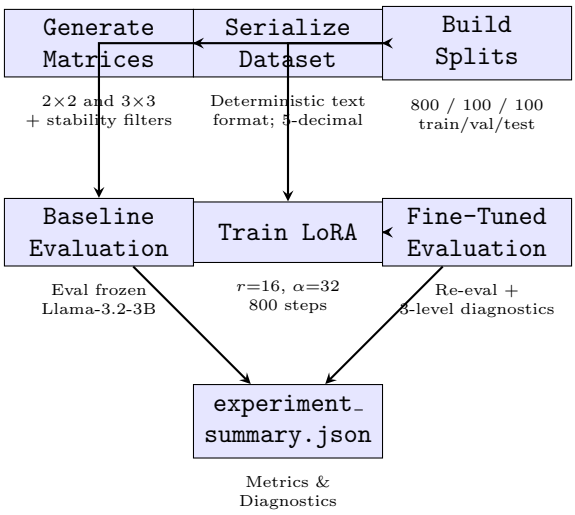


Figure 1: Six-stage MatrixInvLLM pipeline. (1) Generate stable 2×2 and 3×3 matrices with determinant ($|\det A| > 0.05$) and condition number ($\kappa < 50$) filters; 500 examples per size. (2) Serialize to deterministic text format with 5-decimal precision. (3) Split 1,000 examples into 800 train / 100 val / 100 test. (4) Evaluate untrained Llama-3.2-3B baseline on 100 test examples. (5) Apply LoRA fine-tuning (r=16, α=32, 800 steps) on training set. (6) Re-evaluate fine-tuned model using a three-level diagnostic suite: syntactic well-formedness, coarse numeric structure, and residual-based inverse correctness.

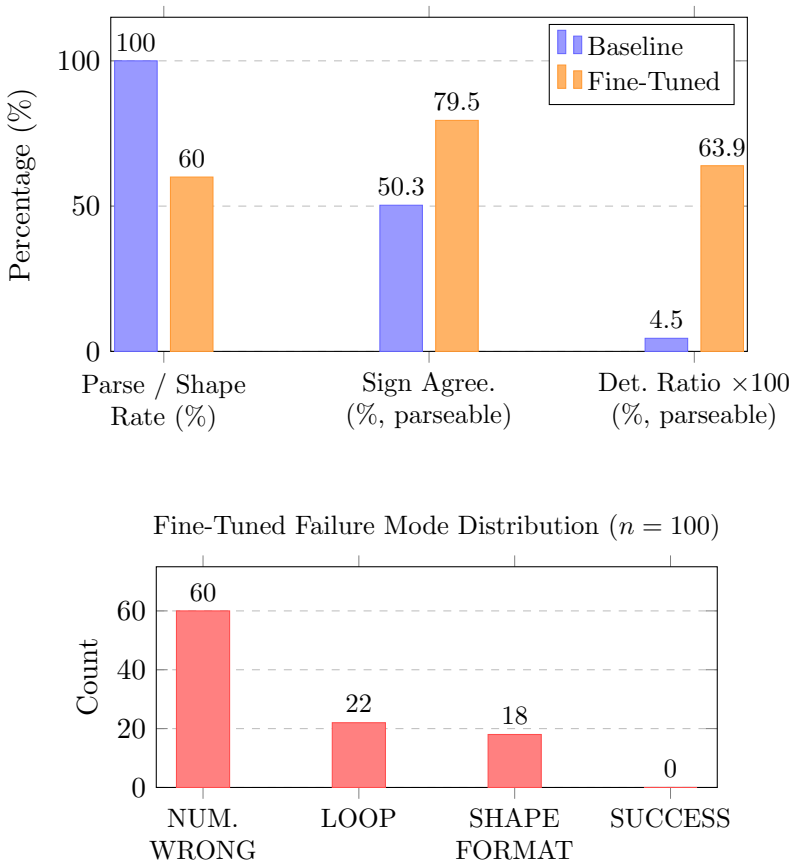


Figure 2: **Top**: Three normalized metrics (all higher = better) comparing baseline and fine-tuned model. Parse/shape rate measures generation reliability; sign agreement and determinant ratio (×100 for scale) measure inverse-like coarse structure among parseable outputs. Fine-tuning improves both structural metrics—sign agreement rises from 50.3% to 79.5%, determinant ratio from 4.5% to 63.9%—but reduces parse reliability by 40 percentage points. Additional diagnostics (residuals, relative Frobenius error, spectral error, copy distances) are in Table 3. **Bottom**: Failure mode distribution for the fine-tuned model ($n = 100$ test examples). 60 outputs are parseable but numerically wrong; 22 loop until the token budget is exhausted; 18 produce malformed shapes; 0 achieve a successful inversion.